

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 8

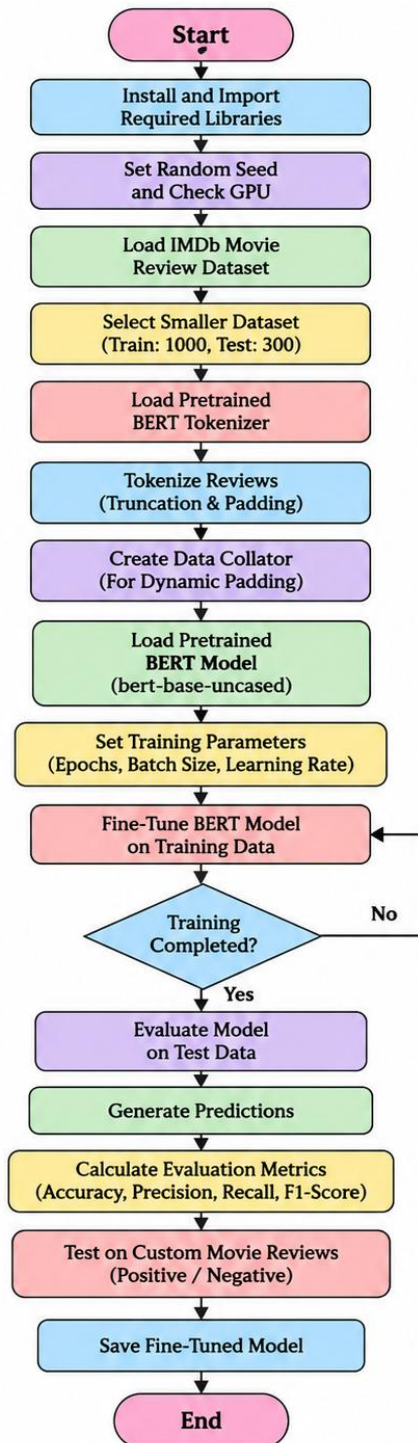
Roll no: 32 PRN No: 12414914 Name of Student: **Rashi Gundawar**

Subject Name : Deep Learning

Problem Statement : Implement Pretrained BERT Model for sentiment analysis or text classification on a sample dataset

Algorithm :

1. Start
2. Install and import the required libraries.
3. Set the random seed and check GPU availability.
4. Load the IMDb Movie Review Dataset.
5. Select a smaller training and testing dataset.
6. Load the pretrained BERT tokenizer.
7. Tokenize the movie reviews.
8. Apply truncation and padding to the tokenized text.
9. Create a data collator for batching.
10. Load the pretrained BERT-base-uncased model.
11. Configure BERT for binary classification.
12. Set training parameters such as epochs, batch size, and learning rate.
13. Fine-tune the pretrained BERT model using the training data.
14. Evaluate the trained model using the test data.
15. Generate sentiment predictions.
16. Calculate Accuracy, Precision, Recall, and F1-score.
17. Generate the classification report.
18. Test the model on custom movie reviews.
19. Save the fine-tuned BERT model and tokenizer.
20. End.



Code :

```
# Install required libraries
!pip -q install -U transformers datasets accelerate scikit-learn

# Import required libraries
import random
import numpy as np
import torch

from datasets import load_dataset

from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    DataCollatorWithPadding,
    Trainer,
    TrainingArguments
)

from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support,
    classification_report
)
```

```
# Check GPU availability

import torch

print("PyTorch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
else:
    print("GPU is NOT available")
```

```
PyTorch version: 2.11.0+cu128
CUDA available: True
GPU: Tesla T4
```

```
# Load the IMDb movie review dataset

dataset = load_dataset("stanfordnlp/imdb")

print(dataset)
```

```
DatasetDict({
  train: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  test: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  unsupervised: Dataset({
    features: ['text', 'label'],
    num_rows: 50000
  })
})
```

```
# Select a small dataset for beginner-level training
```

```
SEED = 42

TRAIN_SIZE = 1000
TEST_SIZE = 300

train_ds = dataset["train"].shuffle(
    seed=SEED
).select(range(TRAIN_SIZE))

test_ds = dataset["test"].shuffle(
    seed=SEED
).select(range(TEST_SIZE))

print("Training examples:", len(train_ds))
print("Testing examples:", len(test_ds))
```

```
Training examples: 1000
Testing examples: 300
```

```
# Display a sample review and its label

print("Review:")
print(train_ds[0]["text"][:500])

print("\nLabel:", train_ds[0]["label"])

print("\n0 = Negative")
print("\n1 = Positive")
```

*** Review:
There is no relation at all between Fortier and Profiler but the fact that both are police series about violent crimes. Pr

Label: 1

0 = Negative
1 = Positive

```
# Load pretrained BERT model

import torch
from transformers import AutoModelForSequenceClassification

MODEL_NAME = "bert-base-uncased"

# Check device
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Using device:", device)

# Load pretrained BERT
model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=2,
    id2label={
        0: "NEGATIVE",
        1: "POSITIVE"
    },
    label2id={
        "NEGATIVE": 0,

```

```
label2id={
    "NEGATIVE": 0,
    "POSITIVE": 1
}
)

# Move model to GPU or CPU
model.to(device)

print("Pretrained BERT model loaded successfully.")

# Load pretrained BERT tokenizer

from transformers import AutoTokenizer

MODEL_NAME = "bert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(
    MODEL_NAME
)

print("BERT tokenizer loaded successfully.")
```

*** Using device: cuda
Loading weights: 100% 199/199 [00:00<00:00, 2406.83it/s]

[transformers] BertForSequenceClassification LOAD REPORT from: bert-base-uncased

Key	Status
cls.seq_relationship.bias	UNEXPECTED
cls.predictions.bias	UNEXPECTED
cls.predictions.transform.dense.bias	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
classifier.bias	MISSING
classifier.weight	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

Pretrained BERT model loaded successfully.
BERT tokenizer loaded successfully.

```
# Tokenize IMDb movie reviews

def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        truncation=True,
        max_length=256
    )

tokenized_train = train_ds.map(
    tokenize_function,
    batched=True
)

tokenized_test = test_ds.map(
    tokenize_function,
    batched=True
)

print("Tokenization completed successfully.")

Tokenization completed successfully.

# Create padding collator for BERT

data_collator = DataCollatorWithPadding(
    tokenizer=tokenizer
)

print("Data collator created.")

Data collator created.
```

```
# Load pretrained BERT for binary classification

id2label = {
    0: "NEGATIVE",
    1: "POSITIVE"
}

label2id = {
    "NEGATIVE": 0,
    "POSITIVE": 1
}

model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=2,
    id2label=id2label,
    label2id=label2id
)

model.to(device)

print("Pretrained BERT model loaded.")

*** Loading weights: 100% ██████████ 199/199 [00:00<00:00, 2466.87it/s]

[transformers] BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key | Status |
-----+-----+
cls.seq_relationship.bias | UNEXPECTED |
cls.predictions.bias | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
classifier.bias | MISSING |
classifier.weight | MISSING |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
Pretrained BERT model loaded.
```

```

▶ # Display number of trainable parameters
trainable_parameters = sum(
    p.numel()
    for p in model.parameters()
    if p.requires_grad
)
print(
    "Trainable Parameters:",
    trainable_parameters
)

```

... Trainable Parameters: 109483778

```

▶ # Define accuracy, precision, recall and F1-score

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(
        logits,
        axis=-1
    )
    precision, recall, f1, _ = (
        precision_recall_fscore_support(
            labels,
            predictions,
            average="binary",
            zero_division=0
        )
    )
    accuracy = accuracy_score(
        labels,
        predictions
    )

    return {
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,
        "f1": f1
    }

```

```

# Set beginner-friendly training parameters
training_args = TrainingArguments(
    output_dir="./bert_sentiment_results",
    num_train_epochs=1,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=16,
    learning_rate=2e-5,
    eval_strategy="epoch",
    save_strategy="epoch",
    logging_steps=25,
    report_to="none",
    fp16=torch.cuda.is_available(),
    seed=SEED
)

```

```

▶ # Create Hugging Face Trainer

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_train,
    eval_dataset=tokenized_test,
    processing_class=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics
)

print("Trainer created successfully.")
# Check whether BERT actually trained

train_result = trainer.train()

print("Training completed.")
print(train_result)

```

```
model=model,
args=training_args,
train_dataset=tokenized_train,
eval_dataset=tokenized_test,
processing_class=tokenizer,
data_collator=data_collator,
compute_metrics=compute_metrics
)

print("Trainer created successfully.")
# Check whether BERT actually trained
train_result = trainer.train()

print("Training completed.")
print(train_result)

Trainer created successfully. [125/125 00:38, Epoch 1/1]
Epoch Training Loss Validation Loss Accuracy Precision Recall F1
1 0.374323 0.366494 0.876667 0.900709 0.846667 0.872852
Writing model shards: 100% [1/1 [00:01-00:00, 1.89s]]
Training completed.
TrainOutput(global_step=125, training_loss=0.5303662109375, metrics={'train_runtime': 39.2518, 'train_samples_per_second': 25.477, 'train_steps_per_second': 3.185, 'total_flos': 131399305490880.0, 'train_loss': 0.5303662109375, 'epoch': 1.0})
```

```
# Evaluate BERT on the test dataset

metrics = trainer.evaluate()

print("Evaluation Results")
print("-" * 40)

for key, value in metrics.items():
    if isinstance(value, float):
        print(f"{key}: {value:.4f}")
    else:
        print(f"{key}: {value}")

Training Loss Validation Loss Epoch Accuracy Precision Recall F1
0.374323 0.366494 1 0.876667 0.900709 0.846667 0.872852

Evaluation Results
-----
eval_loss: 0.3665
eval_accuracy: 0.8767
eval_precision: 0.9007
eval_recall: 0.8467
eval_f1: 0.8729
```

```
# Generate detailed classification report

pred_output = trainer.predict(
    tokenized_test
)

predictions = np.argmax(
    pred_output.predictions,
    axis=-1
)

true_labels = pred_output.label_ids

print(
    classification_report(
        true_labels,
        predictions,
        target_names=[
            "NEGATIVE",
            "POSITIVE"
        ],
        digits=4
    )
)

precision recall f1-score support
NEGATIVE 0.8553 0.9067 0.8803 150
POSITIVE 0.9007 0.8467 0.8729 150

accuracy 0.8767 300
macro avg 0.8780 0.8767 0.8766 300
weighted avg 0.8780 0.8767 0.8766 300
```

```

# Test BERT on new movie reviews
def predict_sentiment(texts):
    model.eval()
    inputs = tokenizer(
        texts,
        return_tensors="pt",
        padding=True,
        truncation=True,
        max_length=256
    )
    inputs = {
        key: value.to(model.device)
        for key, value in inputs.items()
    }
    with torch.no_grad():
        outputs = model(**inputs)
        probabilities = torch.softmax(
            outputs.logits,
            dim=-1
        )
        predictions = torch.argmax(
            probabilities,
            dim=-1
        )
    results = []
    for text, pred, probs in zip(
        texts,
        predictions,
        probabilities
    ):
        results.append({
            "text": text,
            "sentiment": id2label[pred.item()],
            "confidence": float(
                probs[pred].item()
            )
        })
    return results

```

```

# Predict sentiment for sample reviews

sample_texts = [
    "This movie was absolutely fantastic. I loved every minute of it!",
    "The story was boring and the acting was terrible.",
    "It was an enjoyable movie with excellent performances."
]

results = predict_sentiment(
    sample_texts
)

for result in results:

    print("\nText:", result["text"])

    print(
        "Sentiment:",
        result["sentiment"]
    )

    print(
        "Confidence:",
        f"{result['confidence']:.2%}"
    )

...

Text: This movie was absolutely fantastic. I loved every minute of it!
Sentiment: POSITIVE
Confidence: 79.46%

Text: The story was boring and the acting was terrible.
Sentiment: NEGATIVE
Confidence: 80.79%

Text: It was an enjoyable movie with excellent performances.
Sentiment: POSITIVE
Confidence: 87.23%

```

```
# Save the fine-tuned BERT model

SAVE_PATH = "./bert_sentiment_model"

trainer.save_model(
    SAVE_PATH
)

tokenizer.save_pretrained(
    SAVE_PATH
)

print(
    "Model saved successfully at:",
    SAVE_PATH
)

*** Writing model shards: 100% ██████████ 1/1 [00:08<00:00, 8.58s/it]
Model saved successfully at: ./bert_sentiment_model

# Display final assignment summary

print("=" * 45)
print("BERT SENTIMENT ANALYSIS")
print("=" * 45)

print("Dataset : IMdb Movie Reviews")
print("Model : BERT-base-uncased")
print("Classes : Negative / Positive")
print("Training Samples:", TRAIN_SIZE)
print("Testing Samples :", TEST_SIZE)

print("\nEvaluation Metrics:")

print(
    "Accuracy :",
    round(metrics["eval_accuracy"], 4)
)

)
```

```
print(
    "Recall :",
    round(metrics["eval_recall"], 4)
)

print(
    "F1 Score :",
    round(metrics["eval_f1"], 4)
)

print("=" * 45)

*** =====
BERT SENTIMENT ANALYSIS
=====
Dataset : IMdb Movie Reviews
Model : BERT-base-uncased
Classes : Negative / Positive
Training Samples: 1000
Testing Samples : 300

Evaluation Metrics:
Accuracy : 0.8767
Precision: 0.9007
Recall : 0.8467
F1 Score : 0.8729
=====
```